Deploy RAG FastAPI app on azure

Summary flow of the steps to deploy on azure

- GitHub repository contains 19_rag_deploy_version.py, requirements.txt, Dockerfile and permitted PDFs
- Azure PostgreSQL server and demo_deploy are configured
- The vector extension is allowed and enabled
- GitHub Actions successfully pushed the image to Azure container registry (ACR).
- **Azure Container App** has the correct database URL and OpenRouter secret
- Ingress is enabled on port 8000
- /health, /ingest and /ask all work on the Azure URL

Step 1 — Create Github repository with the following structure

Create a github repository named rag_app with the following files

----- 19_rag_deploy_version.py

----- Dockerfile

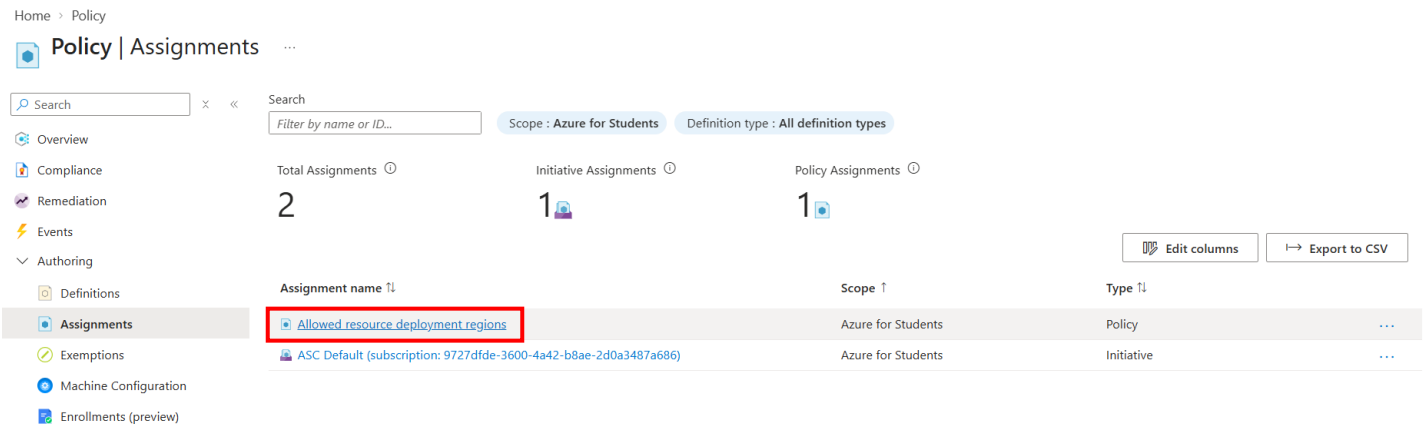
----- requirements.txt

----- .dockerignore

----- pdfs (this folder should have all the pdfs stored to fetch the data from)

Step 2 — Setup your account in Microsoft Azure

- Create an azure account with a valid subscription.
- Login to portal.azure.com
- Check for allowed deployment regions in Policy > Assignments > Allowed resource deployment regions



Parameter ID	Parameter name	Parameter value
listOfAllowedLocations	Allowed locations	["centralindia","eastasia","koreacentral","southeastasia","malaysiawest"]

There are the allowed regions to use in our azure subscription

THE FOLLOWING ARE THE AZURE RESOURCES TO BE CREATED

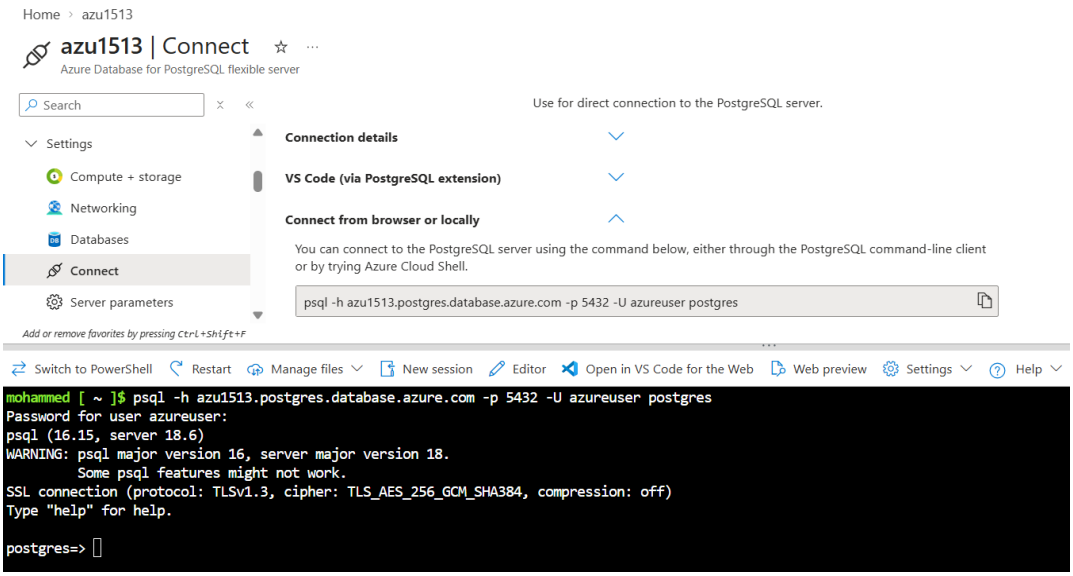
Azure Database for PostgreSQL Flexible Server
Example: `pg-rag-app`
Search for **Azure Database for PostgreSQL flexible servers** → **Create**. Configure the administrator, database, region, and networking.

Azure Container Registry
Example: `acrragapp123`
Search for **Container registries** → **Create**. Use a globally unique registry name and a suitable SKU.

Azure Container Apps
Example: `ca-rag-api`
You'll create this after the container image is built and pushed to ACR.

Step 3 —Configure PostgreSQL and pgvector in Azure

- Create an Azure Database for PostgreSQL flexible server.
- Create a database and a server, set authentication as **PostgreSQL authentication only (put admin login username and password)**
  NOTE : If you add a password with @ in it, then it needs encoding while connecting the database url
- Once postgresql is configured and resource is created,
  go to settings > server parameters > Search extensions > value > search for **vector** and enable it
- Go to settings > databases > create a database **demo_deploy**
- Test connection to postgresql database via azure portal



- In the cloud shell itself, run **\c demo_deploy** -- connects us to the database
- **\dt** -- list all the tables (searches for a table nds_documents, we won't find any table yet)

Step 4 : Create Azure Container Registry (ACR) - Azure builds the container image from your source

Azure Container Registry allows you to build, store, and manage container images and artifacts in a private registry for all types of container deployments.

- Create a container registry with all the default settings and Basic pricing plan

Step 5 — Enable Azure Container Registry access

Open your registry, for example **myragregist123**

1. Navigate to Settings → Access keys.
2. Enable Admin user temporarily for this straightforward GUI-based deployment.
3. Record the login server, username, and one password securely so that we can use it in our next step of Github actions

Step 6 — Configure a remote build through GitHub Actions

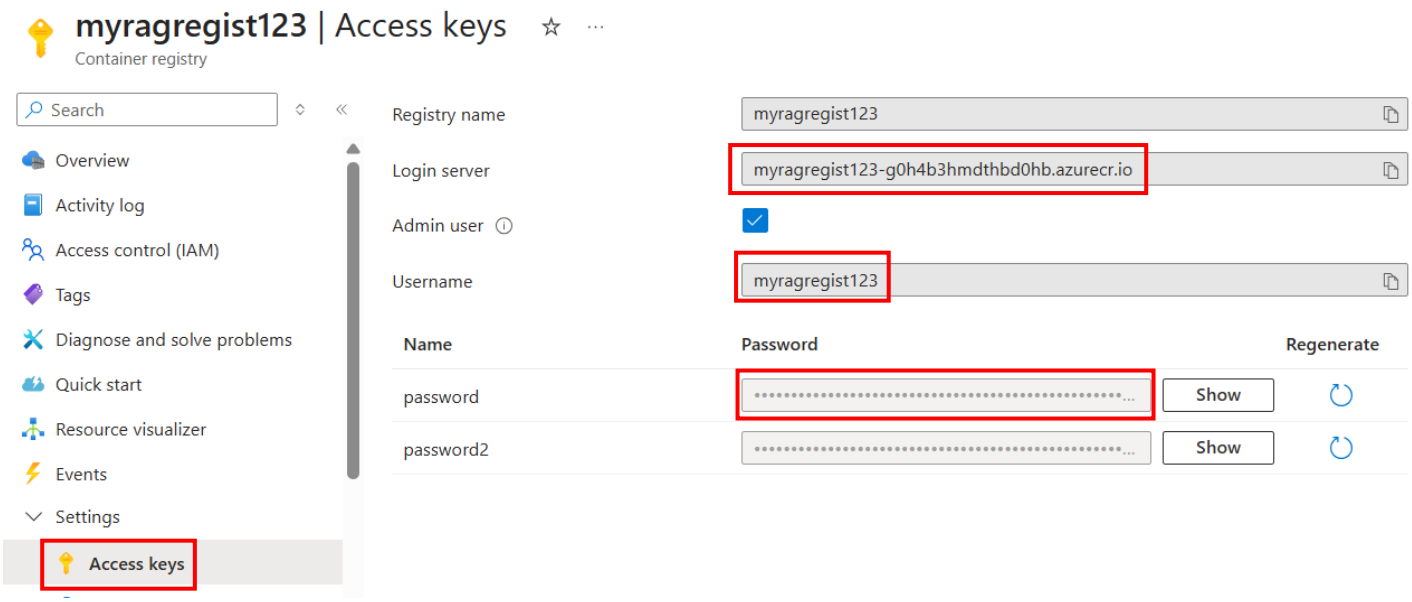
This is the part that replaces Docker Desktop and Cloud Shell. You create the workflow through GitHub's web interface; GitHub's hosted runner builds the image and pushes it into your Azure Container Registry.

6A. Add repository secrets

In your GitHub repository, open Settings → Secrets and variables → Actions → New repository secret.

Add these three secrets:

Secret name	Value
ACR_LOGIN_SERVER	**myragregist123-g0h4b3hmdthbd0hb.azurecr.io**
ACR_USERNAME	**Username shown under ACR Access keys**
ACR_PASSWORD	**Password shown under ACR Access keys**



6B. Create the Github workflow file

In GitHub, select Add file → Create new file. Enter this filename:

.github/workflows/build.yml

Copy the contents of the **build.yml** file.

Commit the file to the main branch. Open the repository's Actions tab and select Build and push RAG API image. Wait for the run to finish successfully.

This workflow creates two tags in ACR: latest and the commit SHA. You can see them under your registry's Repositories → rag-api.

GitHub Actions uses a hosted runner to build the image; you don't have to install or run Docker locally.

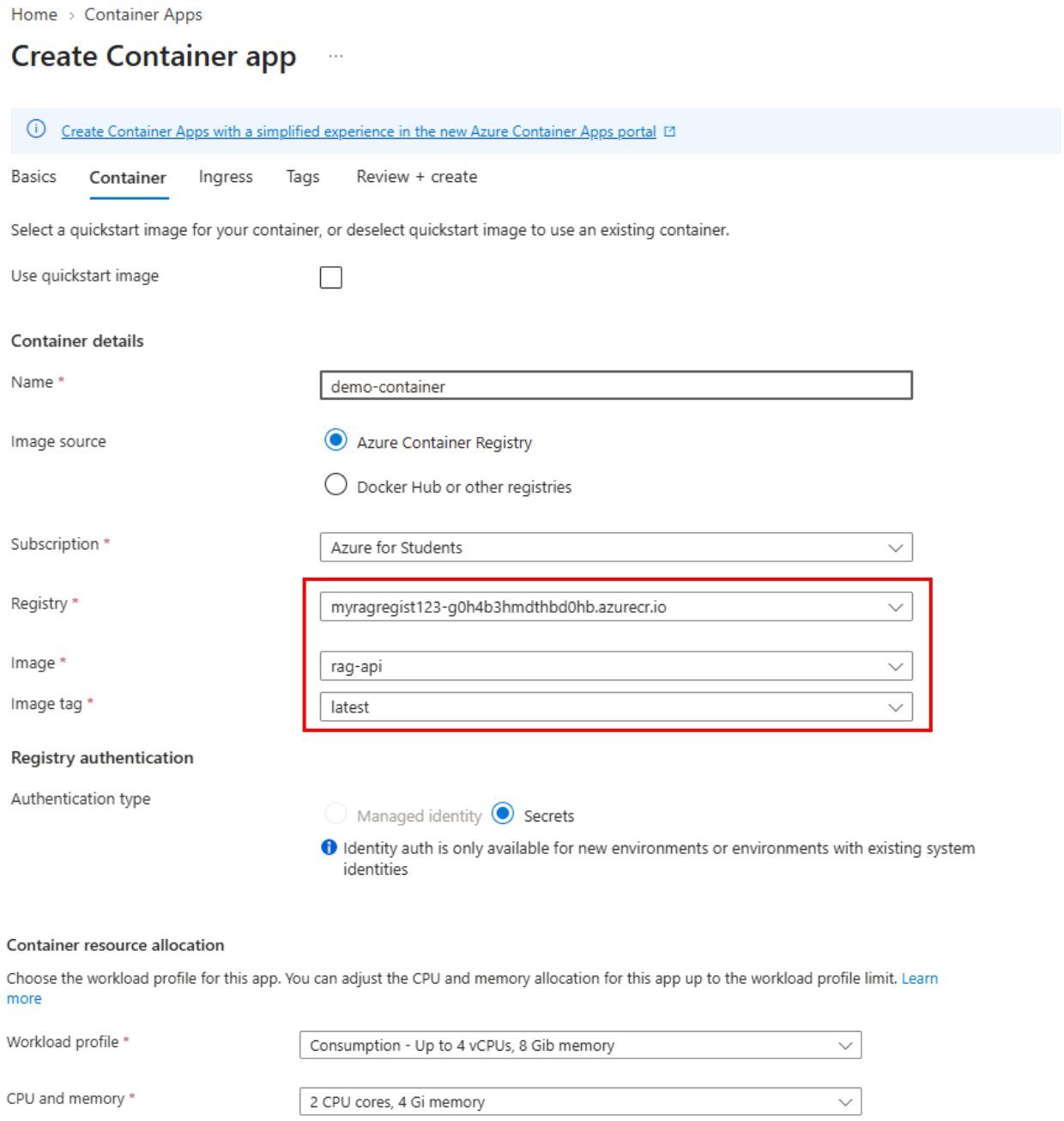
(The Github actions Build and Push step takes about 5-6mins).

Step 7 — Create the Azure Container App

Once the GitHub Actions build succeeds, return to the Azure Portal.

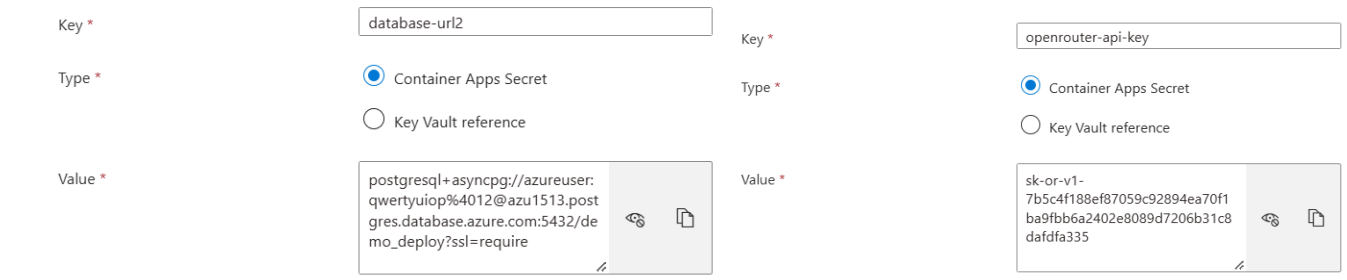
1. Search for Container Apps.
2. Select Create.
3. On the Basics tab, configure:
    - Subscription: your subscription
    - Resource group: Choose your resource group
    - Container app name: ca-rag-api
    - Region: same region as your registry and database, if available
    - Container Apps Environment: create a new environment

4. This step creates a container app by linking the container image stored in the container registry.

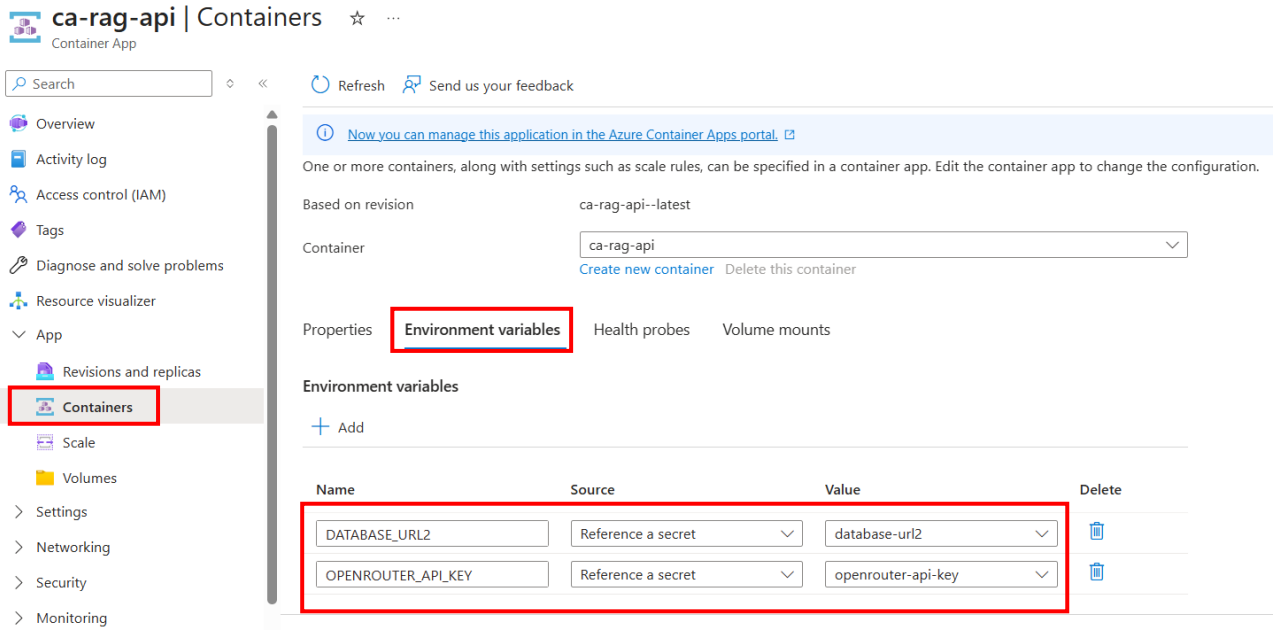


5. Enable Ingress to accept traffic from anywhere and enable target port as 8000.
6. Go to security > Secrets > add 2 secrets **database-url2** and **openrouter-api-key**

Database-url2 :
postgresql+asyncpg://azureuser:password@azu1513.postgres.database.azure.com:5432/demo_deploy?ssl=require



7. Go to App > Containers > Environment variables > Add environment variables and reference it from secrets which we just created.



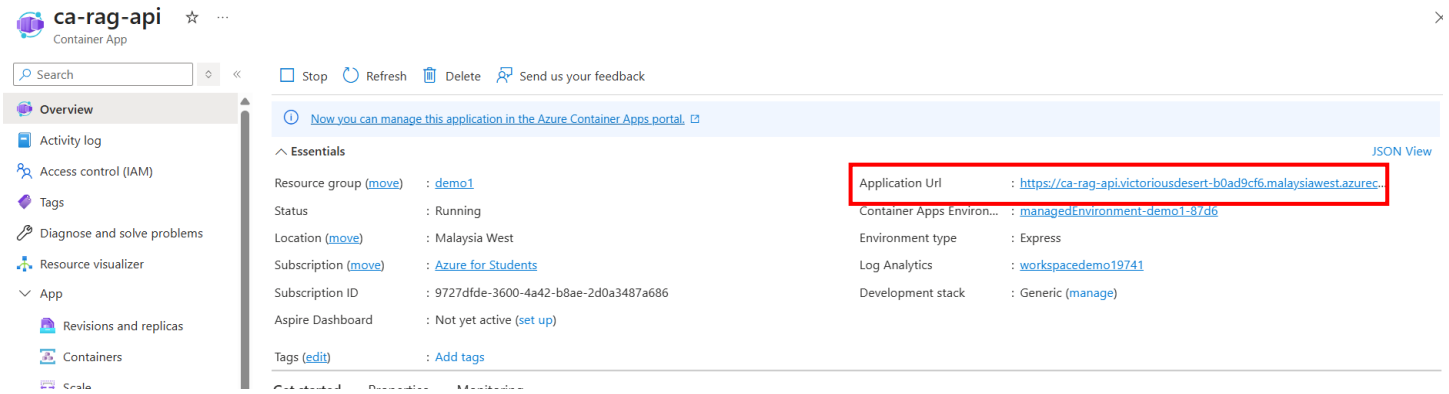
Your deployment is now ready to test

Step 8 — Test the deployed API

After the Container App deployment finishes:

1. Open the Container App resource in Azure Portal.
2. Find Overview and copy the Application URL.
3. Open the URL in your browser, adding /docs.

EXAMPLE URL : https://your-app-name.<your-region>.azurecontainerapps.io/docs



Test these endpoints in order:

1. GET /health — confirms the API responds.
2. POST /ingest — loads the PDFs packaged in the image and inserts the embeddings into Azure PostgreSQL.
3. POST /ask — tests retrieval, reranking, and answer generation.

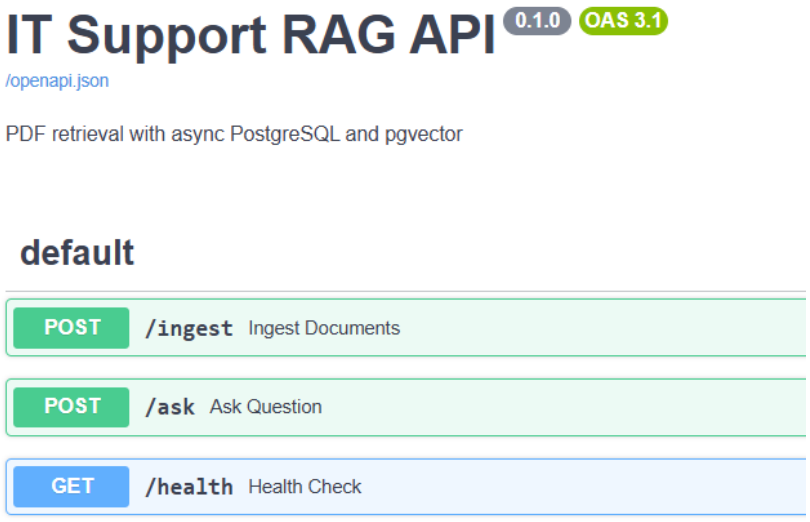
For /ask, send:

{

"question": "How do I troubleshoot VPN connections?"

}

We get the RAG system response retrieved from the code.



Step 9 : To verify and check persistence from the postgresql database

Go to cloud shell

- \c demo_deploy

- \dt #list all tables # should now have a table created

- SELECT * FROM public.nds_documents LIMIT 2; # to view contents of the table

OR

Alternatively can connect to this server via pgadmin and view the results there as well.